

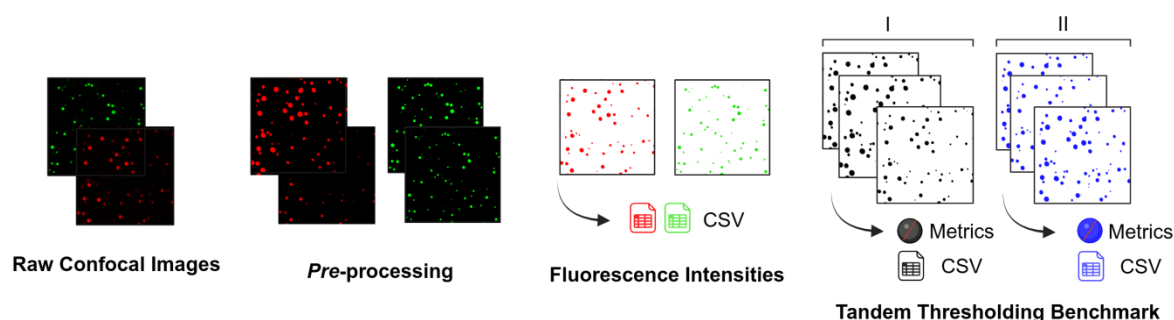
Automated Single-Channel Fluorescence Confocal Image Analysis Workflow for Fiji/ImageJ (1.54p) - SChimanal

Last updated: Aug 13, 2026 (coding in UTF-8),

Readme updated: Aug 13, 2026,

Software version: 1.0.0

Author: Michal Kacper Bialobrzewski



This project is an end-to-end, Python and Fiji pipeline for single-channel fluorescence microscopy (*.lsm*) images. It auto-normalises the selected channel, tests multiple ImageJ threshold models ([doi.10.1117/1.1631315](https://doi.org/10.1117/1.1631315)), extracts morphology metrics (droplets number, area occupied, diameter), plots fluorescence intensity and summary visuals and exports everything to clean outputs with organised file housekeeping.

Who it's for?

Scientists and imaging specialists who need reproducible, high-throughput quantification of fluorescence data-especially in biomolecular condensate studies-and computational biologists who want ready-to-analyse tables and figures without babysitting ImageJ/Fiji.

Overview

This single-file Python workflow automates end-to-end analysis of a selected fluorescence channel from *.lsm* confocal images.

- 1. Fluorescence intensities.** The pipeline first determines the optimal min/max intensity range on the selected channel (C1 by default). Fluorescence intensity is quantified by multiplying each grey-level value by its pixel count and summing the resulting contributions. The final value represents the total fluorescence from all signal-contributing objects visible in the image,
- 2. Thresholding benchmark.** The workflow supports evaluation of any number of Fiji's (19) built-in thresholding models (from just one to all), depending on user selection. Each model is applied to the intensity-adjusted selected channel, which acts as the input mask. To separate merged droplets or overlapping regions, a watershed transform is automatically applied after thresholding and conversion to a binary mask.

For each model, the pipeline generates:

- the original input image
- binary masks resulting from each thresholding model,

- detailed measurement tables

Two parallel analyses are then performed for each model:

- one excluding objects on the edges of the image,
- one including objects on the edges of the image.

For each object, the following metrics are calculated:

- area
- perimeter
- two independent estimates of diameter (one from area, one from perimeter)
- object count

All measurements are saved as **CSV** files, separately for edge-included and edge-excluded analyses. Additionally, all per-model and per-metric statistics are merged and organised into structured **Excel** files.

Metric extraction. These statistical summaries include the median, first quartile (Q_1) and third quartile (Q_3) for every threshold model.

Model rankings. To identify the most reliable thresholding, the pipeline uses a Shewhart-style control chart approach:

- Each metric (Area, diameter, counts) it computes the median across all selected models for two parallel analyses,
- Classify threshold models according to whether all morphology metrics fall within ± 1 SD, ± 2 SD, ± 3 SD, or outside ± 3 SD of the corresponding overall medians.

All masks, overlays, and statistical summaries are saved, both individually and as part of a consolidated dataset. A final comparison plot summarises the thresholds' performance side-by-side, enabling both automatic model selection and manual review.

Output

For every (*.lsm*) image the pipeline spins up a dedicated results folder whose name is a cleaned-up version of the original filename. At the top level you'll find quick-look PNGs (C1 or C2, threshold grid, fluorescence intensities) plus two workbooks that collect all object statistics, and Shewhart scores. Raw data are stored as CSVs.

<working-folder>/

└─ **Channel previews and histograms**

| └─ <base>_original.png # 8-bit, LUT-applied preview

| └─ intensity_distribution_histogram.png

| └─ fluorescence-intensities.xlsx

└─ **Thresholding overview**

| └─ grid_threshold_summary.png # one sheet with all masks side-by-side

| └─ statistics.xlsx # per-model medians with Q_1 and Q_3

| └─ statistics_edges.xlsx # the same, but “objects on edges” only

| └─ statistics_plots.png # 3-panel plot: area, diameter, counts

| └─ summary_threshold_results.xlsx # full per-object tables

| └─ summary_threshold_edges_results.xlsx

| └─ threshold_model_predictions.xlsx # ± 1 SD / ± 2 SD Shewhart ranking

└─ **edges analysis/** # Overlays for thresholding including the edges

| └─ <base>_edges_<Model>.png (× N)

└─ **edges csv/** # CSV tables for the same edge objects

| └─ edges_<Model>.csv (× N)

└─ **csvs/**

| └─ results_<Model>.csv (× N)

| └─ C1 or C2-intensity-histogram.csv

└─ **macros/** # All ImageJ/Fiji macros for full provenance

└─ macro_analysis.ijm

Hardware and System Requirements

The workflow has been tested on Microsoft Windows 10-11, Ubuntu Linux 25.10, and macOS 26.6.1 using Python 3.11.9 and Fiji/ImageJ 1.54p. No specialised hardware or GPU is required. A standard desktop computer with ≥ 8 GB RAM is sufficient.

Installation

Typical installation takes approximately 15-30 min on a standard desktop computer, excluding download time for Fiji and Anaconda.

Setting up Fiji

1. **Install Fiji. Grab the latest bundled Fiji.app** (v. 1.54p at the time of publishing) from (<https://fiji.sc/>). Once installed, run the updater,
2. **Configure Fiji for condensate analysis,**
 - Open original image. In *Set Measurements* tick the following: Area, Std Dev, Min & Max, Area Fraction and Perimeter,
 - *In Analyze* → *Particles* tick the following: Display Results, Clear Results, Summarize, Add to Manager and Overlay.

Setting up Python

1. **Install Anaconda / Miniconda.** Download the latest installer from and follow the on-screen instructions for your OS (<https://www.anaconda.com/>)
2. **Match the project's Python version.** The scripts were developed under Python 3.11.9. Python 3.11.9 is the tested and recommended version.
 - Open an Anaconda terminal and create a 3.11 environment

```
conda install python=3.11.9
```

3. **Update conda in terminal,**

```
conda update conda
```

4. **Install required packages,**

```
conda install -c conda-forge opencv py-opencv pandas pillow matplotlib seaborn scipy openpyxl  
or  
conda env create -f environment.yml  
conda activate confocal-analysis-v1
```

5. **Locate the user configuration section near the beginning of the script and edit.** In version 1.0.0, these settings are located around lines: 223, 1044 and 1045 with the direct path to Fiji.app,

```
Windows: folder_path = "C:/Users/username/fiji-win64/Fiji.app/ImageJ-win64.exe"  
  
Ubuntu/Linux: folder_path = "/home/username/Fiji.app/ImageJ-linux64"  
  
macOS: folder_path = "/Applications/Fiji.app/Contents/MacOS/ImageJ-macosx"
```

6. **Set your working directory.** In version 1.0.0, update the line 1041 to the directory that contains your .lsm images. Only files whose names start with exp are processed, so make sure your images follow that convention,

Windows: `folder_path = "C:/Users/username/"`

Ubuntu/Linux: `folder_path = "/home/username/"`

macOS: `folder_path = "/Users/username/"`

7. **Choose which threshold algorithms to benchmark.** On line 1042 edit the list called *threshold_models*. Provide any subset of Fiji's 19 built-in methods e.g. "Otsu", "Li", "Moments",
8. **Tidy filenames (optional).** If your raw image names carry distracting tags pH, salt, time points, etc. add those fragments to the *shorten_file_name* list on line 41. The helper function will strip them automatically so all outputs stay clean and readable.

How to run

Run the script from the terminal or Spyder program.

```
python github_machine_macro-single_biomolecular-condensates.py
```

All .lsm images in *folder_path* with names that start with *exp* will be processed, and a results folder will be created for each image.

Typical Runtime

Processing one 1024×1024 single-channel .lsm image using all 19 thresholding algorithms takes approximately 1 minute on a standard desktop computer (16 GB RAM, Windows 11).

Demo

Download and extract the example_data and example_output directories. Set folder_path to the example_data directory and run:

```
python github_machine_macro-single_biomolecular-condensates.py
```

Reference demo runtime: approximately 1 min on an Intel i7 desktop computer with 64 GB RAM. The generated numerical outputs should reproduce the reference results deposited in example_output.

Troubleshooting

On systems where Windows uses the CP-1250 (Central European) code page, UTF-8 decoding of Fiji's console output may fail on greek characters e.g. μ etc.

In that case, decode with `encoding="cp1250"` (you can run `chcp` to check the active code page 65001 = UTF-8, 1250 = CP-1250) and keep `errors="replace"` to prevent crashes.

Archived Version

Software version 1.0.0 is permanently archived in RepOD: <https://doi.org/10.18150/2K3PB9>

Acknowledgments

This script and pipeline were designed and written by Michal Kacper Bialobrzewski (<https://github.com/bialobrezuski>). Please cite the following paper when using or adapting this code: <https://doi.org/10.64898/2026.07.16.738963>. I would like to acknowledge Zuzanna Staszalek, Barbara Klepka, Weronika Chmura, Zuzanna Hirszy, and Marlena Stal for their valuable contribution in checking the workflow of the code.